NeuroForge AI

详细使用说明书

2026 年 10 月 24 日

01 从启动到首次训练	安装、工作区、数据与设备
02 从设计空间到最终模型	搜索、代理、模块、Transformer
03 从模型资产到可交付结果	Copilot、生态模型、SDK、论文图
04 日常使用与维护	日志、恢复、备份与升级

随包附 HTML / PDF / Markdown 三种版本与可运行示例。

阅读目录

1. 认识 NeuroForge AI 3
2. 安装、解压与第一次启动 4
3. 桌面启动器与程序退出 6
4. 数据目录、旧版本迁移与存储习惯 7
5. 页面地图和基础概念 8
6. 新手教程：不写代码完成第一次训练 9
7. Dataset Studio：图片与 Token 数据 10
8. Architecture Studio：节点、连接与参数 12
9. Visual Search Space Designer：设计“一组模型” 14
10. Search Lab：算法、预算、候选与回写 16
11. Zero-cost Proxy：运行和解释 17
12. Training Studio：训练参数与设备 18
13. 任务状态、暂停、恢复和检查点 20
14. Tensor & Performance：真正看到网络内部 21
15. Module Library：把子图变成可复用模块 22
16. Transformer / ViT：从模板到可搜索编码器 23
17. Architecture Copilot：配置自己的模型服务 24
18. Optimization Assistant：从目标到实测证据 26
19. Model Ecosystem：导入成熟网络与预训练权重 27
20. Code → Visual 与 Architecture Diff 29
21. Custom Operator SDK：自己的算子进入组件库 30
22. Experiment Lab：比较、记录与导出 31
23. Visual Data Pipeline：预处理与增强 32
24. Export Studio、Figure Studio 与 Neural CAD 33
25. Python SDK、CLI 与 API 35
26. 故障排查：从现象找到处理步骤 37
27. 备份、迁移、升级与卸载 39
28. 三条完整实战路线 40
29. 参数配置与术语速查 41
30. 验收清单、文件索引与参考 43

适用版本：Studio 0.4 / Windows 便携版 0.4.1 文档版本：1.1 • 2026-10-24 • 中文操作手册

本手册带你完成模型设计、架构搜索、训练与结果交付。正文以中文说明操作，并保留界面按钮和字段的英文名称，方便逐步对照。

NeuroForge 支持从源码启动，也支持构建 Windows 便携应用。源码使用 start.bat 或 python run.py 进入工作区；便携应用通过 NeuroForge.exe 启动，并自动打开本机浏览器界面。

阅读路线：第一次使用先读第 1–5 章，再按第 6 章走完首次训练。随后学习搜索空间、模块、Copilot 和模型导入。出现问题时直接查第 26 章。

1. 认识 NeuroForge AI

NeuroForge 将模型结构、数据版本、架构搜索、实际训练和图形输出放进一个本地工作区。你可以用节点搭建 CNN 或原生 Transformer / ViT，检查张量形状，给部分参数定义候选范围，筛选候选后进行实际训练，并导出模型代码、实验记录和架构图。

1.1 产品功能

方向	实际操作入口	可获得的结果
模型设计	Architecture Studio	可执行 Model IR、形状检查、参数与可计算的 MACs、PyTorch 代码
搜索空间	Design / Search Space 切换	参数、重复、分支、条件变量和约束；局部固定、局部搜索
架构搜索	Search Lab	随机、进化、枚举、GP-UCB；真实代理结果、硬件测量和候选快照
实际训练	Training Studio	CPU / 单个 CUDA 设备训练、AMP、梯度累积、检查点、暂停恢复
可观察性	Tensor & Performance	真实激活、梯度、特征图、数值统计、逐节点耗时、整网基准
模块复用	Module Library	子图封装、参数暴露、版本、跨项目插入
AI 辅助	Architecture Copilot	读取上下文、形成结构化计划、编译预览、确认后应用
模型生态	Model Ecosystem	受支持的 torchvision、timm、Hugging Face 模型与权重
自定义算子	Custom Operators	受信任的 Python nn.Module 注册、实测合同、组件复用
数据增强	Data Pipeline	可视化处理链、实际样本预览、参数与版本保存
模型优化	Optimization Studio	非结构化剪枝、CPU Linear INT8、ONNX 导出与推理检查
可视化输出	Export Studio / Figure Studio / Neural CAD	2D 论文图、可编辑图稿、真实 WebGL 3D 场景和 PNG

2. 安装、解压与第一次启动

2.1 两种交付物的区别

交付物	适合谁	启动方式	是否需要外部 Python
Windows 便携应用文件夹	主要使用界面的用户	双击 NeuroForge.exe	不需要；包内包含运行时
源码包	需要改代码、研究实现或自行构建的人	start.bat / python run.py	首次环境配置需要 Python

便携应用包含 Python、PyTorch 与模型适配依赖。将完整文件夹解压到固定位置，双击 NeuroForge.exe 进入工作区。源码用户可以在项目根目录运行 start.bat，或安装依赖后运行 python run.py。

2.2 运行前准备

• 目标平台为 Windows 10 / 11 的 64 位环境。已附带桌面启动器、Python 与所需运行库，无需另外安装 Python 或 .NET。

• 使用现代浏览器。3D CAD 需要浏览器能够创建 WebGL 场景。

• 把整个便携包解压到你可以正常读写的位置。尽量使用较短的路径，例如 D:\Apps\NeuroForge。

• 预留足够磁盘空间容纳解压后的程序、训练权重和数据。具体包大小见随包的 BUILD-MANIFEST / PACKAGE 清单，不应只按压缩文件大小预估空间。

• 可选择 CPU 运行，或在安装兼容 NVIDIA 驱动后选择 CUDA 设备。

• 本地设计和训练不需要模型服务 API Key。只有 Copilot 发出大模型请求时才需要你自己的 Key。

2.3 正确解压后的目录

```
NeuroForge-Windows-0.4.1/
  NeuroForge.exe            双击入口
  START-HERE.txt            简要启动说明
  BUILD-MANIFEST.json       构建和运行时信息
  docs/                     本手册与示例文件
  _bootstrap/               # 桌面入口的运行组件
  _internal/
    app/                    程序、界面和提示词
    python/                 独立 Python 与机器学习依赖
```

_bootstrap 和 _internal 保存便携应用的桌面入口、Python 和机器学习依赖。与 NeuroForge.exe 一起保留即可。自定义 PyTorch nn.Module 可以通过 Custom Operator SDK 注册。

2.4 第一次启动的具体步骤

1. 进入解压后的文件夹，双击 NeuroForge.exe。

2. 等待启动器显示加载本地运行时。首次加载 PyTorch 和初始化数据可能需要一段时间。

3. 看到 **Your workspace is ready** 后，默认浏览器会打开本地地址，通常是 http://127.0.0.1:8787/。

4. 若浏览器没有自动打开，点击启动器的 **Open workspace**。

5. 在网页的 **Runtime** 页面检查 PyTorch、设备列表和数据目录。

6. 初次进入新数据目录时，程序建立示例项目、Shapes-300 图片数据集和 Token Patterns 序列数据集。

7. 先按第 6 章完成小型训练，再导入自己的数据。

2.5 已经有旧服务占用 8787 时

启动器会识别所选端口，连接已有 NeuroForge 服务或提示选择其他端口。Stop backend 用于管理本启动器拥有的后端实例。

需要并行使用独立测试工作区时，可以在命令行执行：

```
.\NeuroForge.exe --port 8877 --data-dir "D:\NeuroForgeData\test"
```

此时打开 http://127.0.0.1:8877/。端口范围为 1024–65535；网页来源校验随所选端口调整。不要手工只改浏览器端口，却不启动对应服务。

3. 桌面启动器与程序退出

3.1 启动器按钮说明

按钮	作用	何时可用
Start backend	启动当前数据目录的本地后端	未连接后端且没有启动操作进行时
Open workspace	打开当前本地网页	后端就绪后
Stop backend	请求安全停止本启动器管理的后端	有控制权且没有活动任务时
Data folder	打开实际数据目录	可用；连接旧服务时打开旧服务的目录
Choose data folder	选择另一份工作区目录	停止当前后端之后
User manual	打开 HTML 版完整手册	docs 目录完整时

3.2 三种“关闭”不要混淆

• **关掉浏览器标签**：不会停止后端，也不会终止训练。

• **关掉启动器但选择 No 保留后台**：后端与任务继续运行。再次打开启动器可以重新连接它。

• **Stop backend / 关闭时选择 Yes**：请求停止后端。有 queued、running、pausing 或 cancelling 任务时，会拒绝停止，保留任务。

结束工作时，打开 Training Studio，选择 Pause 或 Cancel，等待 Paused / Cancelled 状态，再停止后端。Pause 会在当前 Epoch 结束后保存检查点。

3.3 日志、后台控制记录与 API Key

桌面启动日志位于数据目录的 logs/server.log。启动器的本地控制记录也保存在数据目录中，用于重新连接和管理后端。

Copilot Key 的生命周期属于后端进程：仅关网页或者选择保留后台，并不会立刻清空它。使用 **Forget key** 或真正停止后端，才会主动清除会话中的 Key；会话本身最长八小时。

4. 数据目录、旧版本迁移与存储习惯

4.1 程序文件和用户数据是两套东西

新便携版默认数据目录是 %LOCALAPPDATA%\NeuroForgeAI\data，与 EXE 所在位置分开。源代码版默认使用项目中的 runtime。网页 Runtime 页面及启动器 Data folder 显示的是当前实际目录，以那里为准。

```
实际数据目录/
  neuroforge.sqlite3        项目、版本、任务、模块和记录
  datasets/                 图像 / Token 数据与划分清单
  artifacts/                训练检查点、调试 Tensor、优化结果
  models/                   导入模型的本地权重快照与合同
  custom_operators/         已注册算子代码与元数据
  model_cache/              模型服务下载缓存（按适配器而定）
  torch_cache/              torchvision 等缓存
  logs/                     桌面后端日志
```

4.2 从已有源码工作区继续使用

1. 在旧界面确认没有正在运行或排队的计算任务。

2. 停止旧后端；仅关闭网页是不够的。

3. 启动便携 EXE。如果它仍连接旧后端，先关闭旧后端，再点击 Start backend / 重新打开启动器。

4. 在启动器点击 Choose data folder，选择旧版真正的 runtime 文件夹，里面应有 neuroforge.sqlite3。

5. 点击 Start backend，在网页核对项目名称、数据集和历史训练记录。

6. 这样是“打开原工作区”，没有自动复制或合并。后续操作仍会写入该目录。

想先保留一份完全独立的迁移副本，应在旧服务停止后复制整个 runtime 到新目录，再选择新副本。不要把两份数据库或 artifacts 目录直接混合覆盖。

4.3 每天使用的保存习惯

修改节点和搜索空间后按 **Ctrl+S**。长任务开始前检查项目、数据、设备与预算，训练结束后导出实验，并定期备份整个数据目录。

5. 页面地图和基础概念

5.1 页面导航对照

页面	主要回答的问题
Architecture Studio	网络如何连接？形状是否合法？哪些参数可以改？
Search Lab	搜索什么？预算是多少？哪些候选值得继续训练？
Training Studio	任务在哪运行？训练到哪里？能否恢复？
Experiment Lab	某次结果来自哪个架构、数据版本和配置？
Dataset Studio	数据是否导入成功？标签、维度和划分是否正确？
Data Pipeline	数据进入模型前做了什么处理？
Module Library	可以复用哪些结构块？版本和参数是什么？
Tensor & Performance	每个节点实际产生了什么，哪里耗时？
Architecture Copilot	如何把自然语言意图变成可审阅的模型操作？
Model Ecosystem	如何导入已有模型和权重，并使用支持的编辑边界？
Custom Operators	如何把自己的 Python 算子注册成组件？
Optimization Studio	如何生成剪枝、INT8 或 ONNX 结果并做推理检查？
Export Studio / Figure Studio 2.0	如何生成并编辑可交付架构图？
Neural CAD	如何在真实 3D 空间查看、选择和排列模型？
Runtime	当前环境、硬件与数据目录是什么？

屏幕较窄时左侧会缩成图标栏，鼠标悬停可查看名称；标志下方的 **Switch project** 按钮用于切换项目。

5.2 必须理解的对象

Project 是组织入口；**Model IR** 描述一个确定网络；**Search Space IR** 在模型上声明可变参数及约束；**Candidate** 是搜索得到的一个已实例化网络；**Dataset Version** 固定数据和划分；**Job / Run** 是一次实际任务；**Checkpoint** 是训练状态与权重；**Figure / Scene** 是可编辑视觉文档。

改变搜索空间并不自动改变当前固定模型。运行任务时会保存所使用的架构快照；任务不会随着你后续拖动画布而偷偷变化。若要训练一个候选，要明确实例化它，或从候选入口训练该快照。

6. 新手教程：不写代码完成第一次训练

本教程使用内置 Shapes-300 数据，带你快速完成模型设计、代理分析、训练和导出的完整流程。

6.1 建立项目与模型

1. 点击 Switch project，在项目窗口输入 My First CNN，选择 Create project。
2. 进入 Dataset Studio，选择 Shapes-300。
3. 检查它包含三类图形、总样本数和输入尺寸，再点击 **Apply to model** / **Apply to current model**。
4. 回到 Architecture Studio，点击 **Template**，选择一个小型 CNN 或 Residual 模板。
5. 模板会替换当前草稿。尚未保存的修改可以撤销，但开始任务前请先保存。
6. 点击 **Validate graph**。确认界面显示 Graph ready，而不是 Check connections。
7. 点击 Save 或按 Ctrl+S。

6.2 先观察一次真实代理

1. 在 Architecture Studio 点击 Run → Proxy analysis。
2. 选择 cpu。有可用 CUDA 设备时也可选择 cuda:0。
3. 第一轮选择 GradNorm、SNIP；之后再尝试 SynFlow、NASWOT 等。
4. 运行后在 Proxy results / View details 查看状态、设备、耗时和分数。
5. 查看 Completed 状态和各算法的执行记录；选择某个方法可进一步阅读分数、耗时和状态详情。

6.3 启动首次训练

1. 点击 Run → Train model，或进入 Training Studio 创建任务。
2. 设置 Epochs=3、Batch size=16、Learning rate=0.001、Weight decay=0.0001、Seed=42。
3. 选择 CPU 或已经检测到的 cuda:0。首次使用先关闭高级选项，使用 FP32。
4. 检查数据集是 Shapes-300，模型输出为三类，然后确认启动。
5. 观察状态从 Queued → Running → Completed。曲线需要完成首个 Epoch 后才有完整的轮次点。
6. 查看 Train loss、Val accuracy、设备、Epoch、吞吐和检查点信息。
7. 完成后查看 Best validation 和 Held-out test，再去 Experiment Lab 导出该实验。

6.4 成功的判据

你应能找到一个完成状态的训练任务、至少三个 Epoch 的指标、best.pt / checkpoint.pt、对应架构与数据版本，以及导出的实验 ZIP。先追求这个完整闭环；不要因为示例达到较高准确率就推断自己的业务数据也会如此。

7. Dataset Studio：图片与 Token 数据

7.1 图片 ZIP 的正确组织

```
my-images.zip
  cat/
    cat_001.jpg
    cat_002.jpg
    cat_003.jpg
  dog/
    dog_001.jpg
    dog_002.jpg
    dog_003.jpg
```

类别取图片的直接父文件夹名称。不同目录下重复使用同一个父文件夹名，可能被当成同一个类别。推荐让 ZIP 根目录直接包含类别文件夹，不要混入“train/cat”和“val/cat”来期待系统保留原划分；本导入器会按自己的分组规则建立新版本。

7.2 导入配置参考

项目	当前限制 / 行为
压缩包	不超过 100 MiB；解压内容总量不超过 256 MiB
文件数量	不超过 5000 个非目录条目
单张图片	不超过 10 MiB；超大像素图会被列入问题记录
图片类型	PNG、JPG、JPEG、WEBP、BMP
类别	2–100 类，每类至少 3 组内容不同的样本
输入尺寸	16、32、64、128；统一转成 RGB 和所选正方形尺寸
划分	默认约 70% / 15% / 15%，按类别分层并按重复内容分组
重复样本	相同像素内容留在同一划分；跨标签冲突会拒绝导入
原始文件	不就地修改；创建内容哈希标识的新数据资产

导入报告会记录损坏图片、有效样本数和实际分组结果。结合报告与预览确认数据，再保存和应用版本。

7.3 导入与应用的完整步骤

1. Dataset Studio → Import images，填写数据名称并选 ZIP。

2. 选统一尺寸和随机种子，确认导入。

3. 查看样本预览、类别分布、重复数量、问题记录和 train/val/test 数量。

4. 确认标签和图像内容一致，再点击 Apply to model。

5. 返回架构设计器检查输入形状和分类头；通常会更新最后的原生 Linear 输出维度。

6. 如果当前网络是受保护的整体预训练模块，分类合同未必能直接改。请按新数据重新导入合适模型或使用可编辑的分类头。

7. 保存项目，然后启动新的训练任务。旧任务仍引用它自己的旧数据版本。

7.4 Token JSON：导入整数序列

Token 数据使用已编码的等长整数序列。导入前完成分词和长度整理；Token ID 从 0 开始，且小于 vocab_size。

```
[
  {"tokens": [1, 2, 3, 4], "label": "a"},
  {"tokens": [2, 3, 4, 5], "label": "a"},
  {"tokens": [3, 4, 5, 6], "label": "a"},
  {"tokens": [10, 11, 12, 13], "label": "b"},
  {"tokens": [11, 12, 13, 14], "label": "b"},
  {"tokens": [12, 13, 14, 15], "label": "b"}
]
```

在 Import token JSON 中填写记录和词表大小，例如 32。当前支持 6–10000 条记录、长度 1–512、词表大小 2–65536、2–100 类；同类至少三组不同序列。完全相同序列使用不同标签会被拒绝。导入后点击 Use Transformer template / Apply to model，检查输入长度、词表和类别数一致。

8. Architecture Studio：节点、连接与参数

8.1 认识画布的四部分

左侧是组件与模板，中间是模型图，右侧 Inspector 显示选中节点，底部 Console 显示检查与运行反馈。顶部 Design / Search Space 切换编辑模式，PyTorch 标签用于查看生成代码。

节点卡片显示名称、ID、输出形状和参数量。N 表示运行时 batch，图片通常使用 N,C,H,W；序列通常使用 N,L,D。一张 3×32×32 图片不含 batch；实际 batch=16 时输入才是 16×3×32×32。

8.2 添加、连接、布局

- 点击组件会尝试在选中位置后插入；也可拖入画布定位。
- 从一个节点的输出端口，再点目标输入端口建立连接。
- Add / Concat 支持多输入，但相加要形状一致，拼接要满足指定轴以外的维度条件。
- 滚轮缩放，拖动空白平移，拖动节点改变位置。
- Fit canvas 适应视野；Auto layout 整理布局。
- Shift 多选或框选后可封装模块。Ctrl+K 打开快捷节点 / 算子查找。

8.3 改参数时的顺序

选节点 → 在 Inspector 改配置 → Apply changes → Validate graph → 保存。不要只在输入框里改数字就认为它已经应用。修改 Conv 的通道数会影响后续输入；修改 Transformer hidden dimension 会影响 Attention、LayerNorm、位置编码和分类头。

常见字段	含义	典型约束
out_channels	卷积输出通道	后续层的输入通道需要匹配
kernel_size / stride / padding	卷积核、步长和边缘填充	会改变 H、W，残差支路必须保持可相加
out_features	Linear 输出维度	最后分类头应等于数据类别数
embed_dim / hidden_dim	特征嵌入维度	不同算子使用的具体字段以 Inspector 为准
num_heads	注意力头数	必须整除对应的嵌入维度
depth	TransformerStack 等的重复深度	需满足运行预算和输入输出合同
ffn_ratio	FFN 中间层相对宽度	更大通常带来更多参数与计算
dropout / p	Dropout 概率	不同节点配置字段名可能不同

8.4 什么是受保护节点

EcosystemBlock 和 CustomOperator 保存已验证的输入输出合同。可以查看原始模型树，并对图像特征模块使用 Preview searchable replacement，创建合同一致的可编辑替代模块。

8.5 撤销、版本与恢复

Ctrl+Z / Ctrl+Y 对应当前编辑历史；Ctrl+S 保存项目并保留版本记录。Version history 恢复是把历史结构带回画布，并不自动把它对应的训练权重装进所有后续任务。重要实验应同时保存结构版本、数据版本和训练权重。

9. Visual Search Space Designer：设计“一组模型”

搜索空间声明候选变量、取值范围和约束。先让固定模型通过形状检查，再逐步将需要探索的参数设置为可搜索变量。

9.1 一次可执行的 CNN 局部搜索示例

目标：固定输入、池化与分类头，只搜索一个卷积块的通道和卷积核。

1. 在 Architecture Studio 保存当前固定模型。

2. 切换到 **Search Space** 模式，选中目标 Conv Block。

3. 在 Inspector 的 Search Space 面板点击 **Add search variable**。

4. Parameter / structure 选择 config.out_channels，Domain type 选 Discrete choices，填 [8,16,32]。

5. 保存变量，再添加 config.kernel_size 的候选，例如 [3,5]。

6. 检查 padding 是否和 kernel_size 共同满足输出尺寸要求；固定 padding 不一定适合所有卷积核。必要时一并定义变量或选择保持形状的块。

7. 打开 Search Space Summary，检查理论组合数、合法组合数和过滤原因。

8. 设置足够容纳这些模型的参数、MACs 和预算，保存后进入 Search Lab。

这组变量的笛卡尔积最多是 3×2=6，但实际合法、不同的架构可能更少。过滤、条件分支和重复去除都会改变数量。

9.2 Fixed、Searchable、Conditional

• **Fixed**：该节点参与当前固定模型计算，但它的相关变量不参与这次搜索。

• **Searchable**：在定义的候选域中取值。

• **Conditional**：满足指定变量依赖时才取候选值；不满足条件时使用默认 / 基础配置。

局部搜索 Head 时，将变量添加到 Head 或其模块内部，并让 Backbone 保持 Fixed。使用 Summary 核对参与搜索的变量和节点。

9.3 变量编辑器字段对照

字段	应怎样填写	常见错误
Parameter / structure	config.参数名，或 repeat / enabled / op / module	选了并不存在的配置字段
Discrete choices	JSON 数组或逗号分隔标量，如 [16,32,64]	把表达式、Python 列表推导式当作值
Numeric range	Min、Max、Step；选择 Integer / Float	Step=0、Min 大于 Max、整数取整后重复
Sampling	Linear 或对数采样	对数下界必须为正
Boolean switch	false / true	把字符串 "false" 与布尔 false 混用
Depends on variable	先选一个已经存在的变量	依赖自己或形成循环

字段	应怎样填写	常见错误
Conditional value	与依赖变量比较的标量	候选实际是数字，条件却填成字符串
Default value	条件未启用时采用的合法值	默认值本身导致张量冲突

每个变量最多 64 个离散 / 采样值；线性范围也最多生成 64 个值。整个空间最多 64 个变量。重复次数变量使用 0–24 的整数；重复或绕过要求边界满足相应单输入和形状约束。

9.4 结构变量的注意事项

repeat 控制节点或模块的重复次数，enabled 控制分支启用，op 选择兼容算子。模块替换使用模块库中已经检查的定义；配置后运行形状检查，确认周边连接与输出维度。

9.5 全局与局部约束

全局约束可限制参数量、MACs/FLOPs、层数、静态内存，以及输入输出形状；局部关系可表达可整除、相等、不大于或不小于。界面中的 Params / MACs 字段可能以 M 为单位，填 50 代表约五千万；IR / API 中对应 max_params=50000000。请区分 UI 显示单位与底层整数。

在 Search Lab 的 Algorithms & hardware 中开启硬件测量，配置延迟、峰值显存或吞吐条件。候选结果会记录目标设备的实测值。

受保护模型内部无法计算完整静态 MACs/FLOPs。此时不要用部分可见层的数值充当整个模型成本；应去掉不适用的显式静态成本约束，改用实际硬件指标。Summary 会显示具体拒绝原因。

10. Search Lab：算法、预算、候选与回写

10.1 怎样选择搜索策略

策略	实际含义	适合的第一次使用方式
Random	在合法候选域中随机取样	先做 8–16 个候选的基线
Evolution	从已有较好候选出发做结构 / 参数变异	空间稍大且希望继续探索邻域
Enumerate	遍历有限离散空间并检查	小空间、希望看清每个合法组合
Bayesian / GP-UCB	用已有候选信号建立 GP 代理并提出下一批	变量不多、每次代理较贵，且已有可用数值信号

Bayesian 根据当前搜索采用的代理或评分信号更新模型，通过 GP-UCB 选择下一批候选。配合实际训练结果，逐步优化架构选择。

10.2 启动搜索前的核对清单

确认：当前项目正确、数据版本正确、模型能编译、至少有一个活动搜索变量、候选预算合理、资源上限没有把所有候选排除、设备确实存在、所选代理至少有一种适用于当前网络。先用小候选数验证闭环，再扩大预算。

每次搜索可配置 2–256 个候选，以及整个任务的时间预算。系统根据合法组合、去重、约束和剩余预算确定实际探索数量。

10.3 Hardware-aware 搜索

在 **Algorithms & hardware** 中选择代理，勾选 **Measure target device performance**，选择目标设备，按需要设置 Max latency / batch、Max measured peak、Min samples / second。

比较延迟时，将 batch size、输入形状、精度、设备和重复次数一起保存。统一测量协议后，可在候选之间比较延迟、吞吐与显存。

10.4 怎样阅读候选结果

- 综合排名融合本次执行的代理信号；同时展示各算法原始分数和归一化排名。
- 查看每个代理是否 Completed、是否有不可用原因，以及不同代理的分歧。
- Pareto 候选展示规模、评分和硬件指标之间有代表性的取舍。
- 同时观察模型大小、实际延迟、吞吐和峰值，避免仅凭一个高分选择超预算模型。
- 选择两名候选查看 Architecture Diff，确认模型究竟改了什么。

10.5 三种后续操作

Train candidate：直接基于候选快照训练。**Instantiate as new version / Open on canvas**：把候选变成固定模型，回到 Studio 保存并继续编辑。**Save candidate snapshot**：创建独立项目副本，保留该模型作为分支起点。

回写固定模型后，其搜索变量通常不会继续保持原模板的可搜索状态；需要继续搜索时，应重新确认空间是否存在。比较“模板→候选”的 Diff，可以避免误把一个经过实例化的固定模型当成原搜索模板。

11. Zero-cost Proxy：运行和解释

11.1 各种代理主要看什么

方法	观察信号	读结果时应注意
SNIP	权重与损失梯度的敏感度	需要前向、反向和数据标签
SynFlow	网络信号流 / 权重路径	查看初始化、分数和实际执行配置
NASWOT	ReLU 激活模式核	查看核统计、有效性和分数
GradNorm	梯度强度	与数据批次、损失和初始化有关
ZiCo	梯度统计变化	批次与梯度统计不充分会影响有效性
Fisher	与梯度 / 敏感性有关的统计	需要看实现状态和采样上下文
Jacobian Covariance	输入 / 输出相关导数的结构信号	对网络结构、输入类型和数值状态有要求

Params / MACs / FLOPs 展示结构规模。比较代理算法时，使用各自的原始分数、归一化排名和同一评价配置。

11.2 一次规范的代理对比

固定数据版本、随机种子、采样方式和设备，对相同候选集合使用一致的方法。查看原始分数、状态、时间、显存、batch 和初始化方式；导入模型使用其记录的权重快照。

Proxy analysis 分析当前模型，Search Lab 则对生成的候选进行约束筛选。选中代理记录可以查看执行状态和结果详情。

11.3 排名之后做什么

选择一组有代表性的候选进行短训练，比较验证表现和成本，再为最终候选分配完整训练预算。保存代理与训练记录，便于复用选择经验。

12. Training Studio：训练参数与设备

12.1 参数参考表

参数	默认或范围	操作说明
Epochs	默认 5；1–200	先小轮数验证流程，再按任务调整
Batch size	默认 32；2–128	是微批次；过大可能 OOM
Learning rate	默认 0.001；1e-6–0.1	结合优化器、batch 和任务调整
Weight decay	默认 0.0001；0–1	正则化配置，和实验一起保存
Seed	默认 42	记录初始化与数据处理的随机配置
Device	auto / cpu / cuda:N	auto 自动选择可用设备；也可指定 cuda:N
AMP / FP16	CUDA 下生效	CPU 下不执行 CUDA AMP；检查实际生效状态
Gradient accumulation	1–32	多个微批次后执行一次优化器更新
Early stopping patience	0–50，0 表示关闭	连续若干轮无足够验证提升后停止
Minimum improvement	0–1	验证改进判据，需和 patience 一起理解
Auto batch size	可选	在 OOM 预检中降低微批次，并保存实际生效的微批次

训练采用 AdamW、CosineAnnealingLR、梯度裁剪和 Epoch 检查点。按数据、模型与设备设置参数，并用短训练检查配置。

12.2 AMP 和梯度累积的实例

例如使用微批次 8、accumulation=4，可以形成约 32 个样本的优化器更新规模。BatchNorm 按实际微批次工作；运行记录会保留微批次、AMP 和累积步数，方便比较。

Auto batch 预检发现 OOM 时会尝试减小微批次。它不保证将梯度累积自动改成等效原 batch；完成后一定查看 effective_batch_size 和 effective_accumulation。

12.3 训练前必须确认的数据合同

模型输入应与所选数据 shape 一致，最后输出应等于类别数。图像分类的最后输出应为 [类别数]（不含 N），不要额外把 Softmax 当成必须添加的训练层；当前训练使用交叉熵处理分类 logits。

训练候选模型时使用其所属任务的数据与处理快照；原生固定模型使用提交时的当前架构快照。查看任务配置可确认本次训练的具体输入。

12.4 监控面板怎样读

Train loss 来自训练过程，Val loss / accuracy 来自验证集。Best validation 可能属于较早轮次；最终测试通常使用最佳验证权重，而不是最后一次优化器更新后的参数。

Training Studio 展示训练协议下的吞吐、Step time、Epoch time 和显存计数；Profiler 展示推理协议下的延迟与 samples/s。比较时使用相同的设备、batch 和测量设置。

13. 任务状态、暂停、恢复和检查点

13.1 状态机

```
Queued -> Running -> Completed
                  -> Failed
                  -> Pausing -> Paused -> Resume -> Running
                  -> Cancelling -> Cancelled
```

Queued 表示等待单个本地计算 Worker。Running 说明已经开始执行；Pausing / Cancelling 是请求已发出但当前操作尚未完成。不要在这两个中间状态时直接结束后端并期待得到最新检查点。

13.2 暂停并恢复训练

1. Training Studio 找到目标任务，点击 Pause。

2. 等当前 Epoch 完成和检查点写入，确认状态为 Paused。

3. 可以继续看已有曲线和日志，但不要移动正在引用的数据目录。

4. 点击 Resume training / Resume from checkpoint。

5. 查看日志中的恢复 Epoch。程序恢复模型、优化器、调度器、RNG 和 AMP scaler 等状态。

若任务在第一轮完成前就被强制终止，可能没有可恢复检查点。服务器重启发现未正常结束的任务会保留记录并转为暂停 / 中断相关状态，但这不等于存在任意 step 级别的恢复点。

13.3 best.pt 与 checkpoint.pt

文件	用途	不应怎样使用
best.pt	验证表现较好时的模型 state_dict	不包含完整优化器恢复状态
checkpoint.pt	最近完整 Epoch 的训练状态	不要把它当作纯 state_dict 直接传给 load_state_dict
model.py	与对应 IR 一致的模型构造代码	不应与另一个版本的权重混用
architecture.json	架构结构数据	单独持有它不会自动恢复权重和数据

Cancel 与 Pause 的目的不同。取消表示结束这次执行，不承诺按照暂停入口继续；重新运行通常是创建新任务。失败日志与已存在的检查点会保留，不要因为 Failed 就先删除目录。

14. Tensor & Performance：真正看到网络内部

14.1 Tensor Debugger 操作步骤

1. 在 Studio 选一个节点 → Tensor & gradients，或打开 Tensor & Performance。

2. 点击 **Capture tensors**，选择设备、初始化权重或完全匹配架构的训练检查点。

3. 选择训练集样本数（inspect batch 通常 2–16）和种子，再启动。

4. 在左侧节点列表选择要观察的节点，切换 Activations / Gradients。

5. 看 shape、均值、标准差、范围、零值比例、非有限数值、直方图和原始预览。

6. 对图片 Tensor 选择 Feature channel；需要完整数值时下载 NPZ。

调试器执行真实前向和交叉熵反向，不会因为查看节点而执行优化器更新。Token 输入是整数 ID，本身没有可微梯度；Embedding 后的浮点特征仍可能有梯度。没有梯度也可能是不可微输入、未参与输出的张量或算子行为所致，不能一概解释为训练失败。

14.2 特征图和数值预览的范围

界面展示首个样本的通道预览，较大特征图采用平均下采样；数值面板展示原始值片段。每任务约 64 MiB 的 NPZ 捕获预算用于保存可下载数组。

14.3 Profiler 与图上覆盖

点击 **Run profiler** 选择模型、设备、权重和重复次数。它使用实际 CPU 时间 / CUDA events 记录节点开销，并做 warmup 后的整网 benchmark。Graph overlay 会把匹配架构的耗时标在设计图上；架构改变后旧测量不会自动成为新测量。

模块耗时包含其内部执行，逐层查看时按层级阅读。Activation bytes 记录输出大小，allocator delta 记录分配变化，整网 benchmark 展示统一协议下的模型延迟。

15. Module Library：把子图变成可复用模块

15.1 封装模块的完整操作

1. 在画布用 Shift 多选相关节点，或 Shift 拖动框选。

2. 右键选择 **Create Module**，或点击顶部封装图标。

3. 输入 Module name、Category 和 Description。

4. 检查自动识别的边界输入、输出。

5. 选择要暴露的内部配置，例如 channels、num_heads、ffn_ratio。

6. 点击 Package & save to library。

7. 看到模块卡片后，双击 / Expand 进入内部，检查结构仍然正确。

8. 返回完整模型，验证并保存。

当前封装面向具有清晰边界的子图，要求一个可识别的输出边界。选中互不相连的零散节点、多个独立输出或缺失外部输入，可能会被拒绝。先明确“模块是从哪个 Tensor 读入、向哪个 Tensor 写出”。

15.2 暴露参数意味着什么

暴露的是外部配置到内部字段的绑定，例如模块的 width 绑定内部某个 Conv 的 out_channels。外部参数改变后仍要通过编译器检查；暴露一个维度并不会自动替你发明所有相关投影或残差连接。

可以给模块外部配置添加搜索变量，也可进入内部只搜索部分节点。不要同时对同一字段建立互相冲突的外部和内部变量；保持参数绑定关系清晰。

15.3 版本与复用

在另一个项目进入 Module Library，按分类或搜索找到模块，插入后检查它与当前位置的输入 shape 是否匹配。收藏和最近使用帮助查找，不改变计算语义。修改后可以 Save module version，保留旧版本以便复现。

模块 JSON 适合传递结构。若模块引用受保护模型或自定义算子资产，仅传 JSON 可能不包含所需本地文件；对外交付更适合使用包含资产的项目 / 实验导出，并让接收方安装相应运行时。

16. Transformer / ViT：从模板到可搜索编码器

16.1 原生 ViT 的组成

Image Input → PatchEmbedding → ClassToken（可选）→ PositionalEmbedding → TransformerStack → LayerNorm → TokenPooling → Linear classifier。原生模板是可编辑 IR；它不同于导入后的受保护 timm / HF backbone。

32×32 图片配合 patch size=4，得到 8×8=64 个 patch；启用 class token 后序列长度为 65。嵌入维度 32 可搭配 heads=4、8、16。形状检查会验证残差、位置编码和归一化维度。

16.2 复现一个局部搜索案例

1. 从 Template 选择原生 ViT，先确保能验证通过。

2. 找到 TransformerStack（常用 ID 为 encoder），切到 Search Space。

3. 给 depth 设置 Numeric range：Min=4、Max=12、Step=1、Integer。

4. 给 num_heads 设置 Discrete choices：[4,8,16]。

5. 固定 PatchEmbedding、位置编码、归一化与分类头；不要额外开放 hidden dim。

6. 参数量上限设置为 50M。

7. Summary 应显示理论 9×3=27 个组合；合法数取决于当前模型合同。

8. Search Lab 使用 Bayesian / GP-UCB，目标设备选 cuda:0，并开启实际硬件测量。

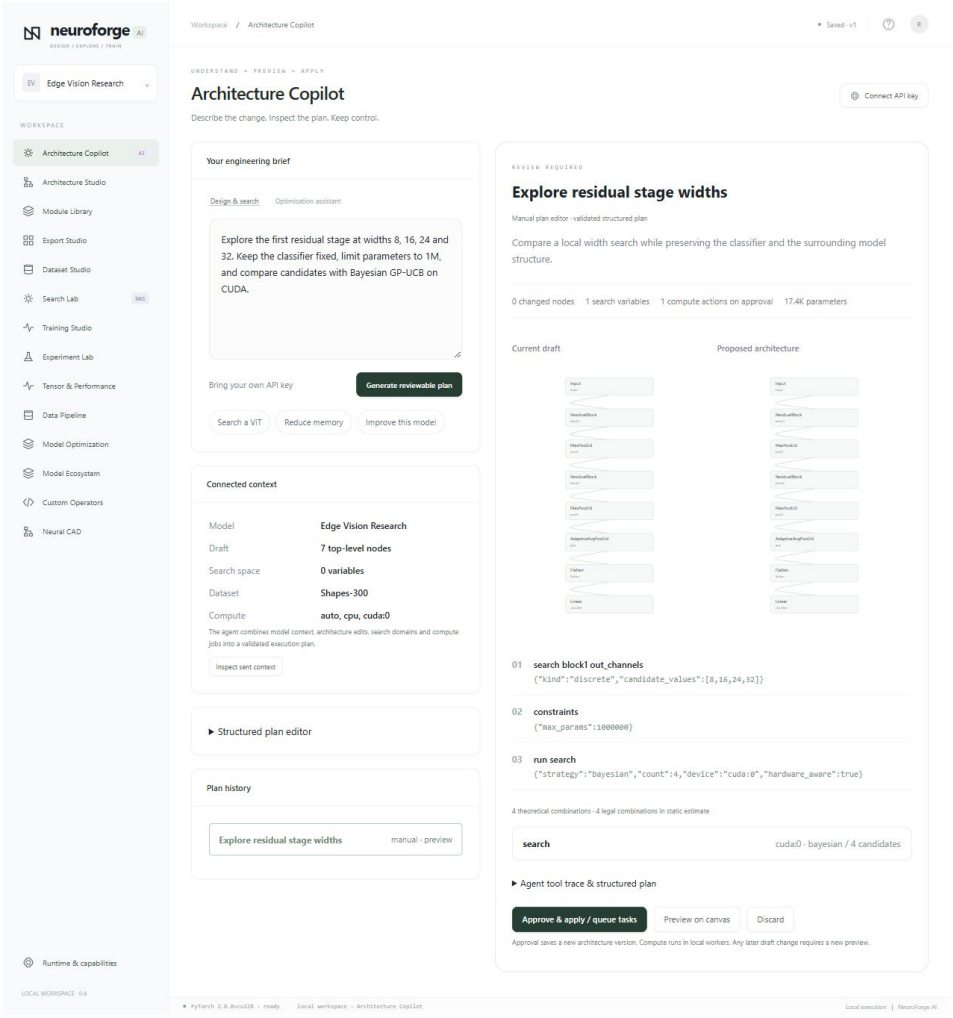
9. 查看 Summary 中的 27 个唯一组合，按空间规模安排候选预算。

10. 选择若干候选短训练，比较实际验证表现后再决定完整训练。

16.3 Token Transformer 的额外核对

先准备等长 Token ID 序列，并统一词表、序列长度、Embedding 维度和类别输出。使用 BERT / GPT-2 序列分类适配器时，按其配置准备词表和输入。

17. Architecture Copilot：配置自己的模型服务



架构计划预览

17.1 配置的操作步骤

1. 打开 Architecture Copilot，点击 **Connect API key**。

2. 选择协议：Responses API 或与本工具调用格式兼容的 Chat Completions。

3. API base URL 填服务的基础地址，不要把 API Key 放在 URL 的 query、fragment 或用户名密码里。

4. Model ID 填你实际拥有访问权限、支持函数工具调用的模型标识。程序不会为你购买模型访问或自动选择最贵模型。

5. 在独立 API key 密码框输入自己的 Key，保存配置。

6. 用一个小请求，例如“解释当前模型并预览一个不启动计算的局部改动”，检查服务是否可用。

7. 若 HTTP 401 / 403 / 404、连接失败或没有 tool calls，按第 26 章排查。

OpenAI Responses 常用的基础地址为 https://api.openai.com/v1。产品代码会根据协议追加 /responses 或 /chat/completions，不要在基础地址中重复追加完整请求路径。兼容服务必须支持所发送的函数工具和 JSON schema；“支持聊天接口”不必然意味着支持这个 Agent。

17.2 Key 与发送数据

Key 只保存在服务端 RAM 中，最多八小时，浏览器收到 HttpOnly 会话 Cookie；不会写进项目数据库、localStorage 或模型导出。Forget key 与停止后端会清理会话。关闭网页或保留后台进程不等于停止后端。

生成计划时，所选服务会收到用户请求、当前 IR、搜索空间、数据集元信息、设备和聚合测量结果，不发送图片样本、训练 Tensor 或权重文件。用 **Inspect sent context** 查看将被使用的上下文。模型结构本身也可能是你的研发资产，选择服务前应自行确认是否允许发送。

17.3 怎样描述任务更容易得到可执行计划

明确“修改范围、保持不变的部分、参数候选、硬约束、设备和计算预算”。例如：

Keep the patch embedding and classifier fixed. Make the encoder depth searchable from 4 to 12, with heads 4, 8 or 16. Limit parameters to 50M. Preview a Bayesian GP-UCB search of up to 100 candidates on cuda:0 with measured latency.

中文也可以作为用户请求，由你配置的模型理解；界面和生成计划说明使用英文。若请求同时包含替换结构、改数据、训练多个阶段和比较精度，先拆成“建立基线”“改动并训练”“比较结果”三个可审阅阶段。

17.4 计划预览具体检查什么

在预览中检查 Before / Proposed architecture、节点、参数、搜索变量、约束、设备、候选预算和任务配置。确认后执行，随后在候选和训练记录中查看结果。

点击 **Preview on canvas** 可以在当前画布看到待修改位置提示；草稿在确认前保持原样。需要真正执行时点击 **Approve & apply / queue tasks**。它会保存一个新版本并排队对应任务。Discard 丢弃待确认计划。

17.5 为什么计划可能过期或失效

计划审批有效期为一小时。修改草稿、搜索空间、数据或项目版本后，重新生成预览；Plan history 保存各次计划和状态。

17.6 无 Key 时仍可使用什么

Structured plan editor 用于编写并验证结构化计划。填写 JSON 后点击 Validate & preview，审阅图变化和任务配置，再点击 Approve & apply 执行。

18. Optimization Assistant：从目标到实测证据

选择 Copilot 中的 **Optimization assistant**，描述降低显存、缩短延迟或减小模型的目标。系统结合配置、图结构、局部搜索和 Profiler 生成优化计划，再用训练和评测记录比较效果。

18.1 三阶段优化工作流

阶段 A：基线。 保存当前模型，使用目标设备和固定数据记录真实训练或 Profile；保存权重与配置。没有基线时先执行只测基线的计划。

阶段 B：改动。 例如启用 AMP、降低微批次并提高梯度累积，或替换一个可支持的 Stage。预览并确认后执行匹配训练。注意一个计划里的任务使用的是该计划最终图，不会自动把某个任务当成修改前的图。

阶段 C：比较。 在 Verify observed optimization 中选 Baseline run 和 Optimized run，再点击 Compare real memory & accuracy。需要同一数据、处理流程、种子、Epoch、设备、环境和等效优化器 batch；不匹配时不应判定目标已达到。

18.2 怎样理解比较报告

默认目标是峰值显存下降 30%，测试准确率变化控制在 1 个百分点以内。报告并排显示峰值显存、减少比例、精度变化、微批次和 AMP 状态，便于判断优化效果。

例如显存从 75.84 MiB 降到 68.13 MiB，报告会显示约 10.16% 的下降幅度。继续调整微批次、AMP 或局部模块，再用相同的比较流程观察下一轮结果。

19. Model Ecosystem：导入成熟网络与预训练权重

19.1 模型目录

来源	已整理的模型	导入后怎样展示
torchvision	ResNet-18 / 34 / 50	Stem、各 Stage、池化和分类头边界；Stage 内部保留真实 PyTorch 模块
torchvision	EfficientNet-B0、ConvNeXt-Tiny、ViT-B/16、Swin-T	受保护 Backbone，加任务分类头；原始模块树只读查看
timm	ResNet-18、EfficientNet-B0、ConvNeXt-Tiny、ViT-Tiny、DeiT-Tiny、Swin-Tiny	真实特征提取 Backbone 与新分类头
Hugging Face	BERT、GPT-2 分类、ViT、Swin 适配器	真实模型资产、配置与权重快照

选择模型后，导入器会检查当前数据、输入尺寸和显存需求。BERT 与 GPT-2 通过序列分类适配器使用 Token 数据，ViT 与 Swin 通过图像分类适配器使用图片数据。

19.2 一次标准导入

1. 先创建项目并应用数据集，核对输入尺寸与类别数。

2. 打开 **Model Ecosystem**，选择 Provider 和 Model。

3. 不需要预训练能力时，先不勾选 Pretrained；这会保存初始化后的真实模型快照。

4. 需要公开预训练权重时，启用对应选项并明确允许 Download。没有下载授权不会静默获取权重。

5. 启动导入任务，观察日志。首次导入可能需要下载配置或较大权重，不能把等待误判为画布卡死。

6. 任务完成后检查 Weight status、Input / Output contract 和 Notes，再将导入结果应用到当前项目。

7. 保存项目，先做一次小批次 Profile 或训练，确认数据与设备兼容。

预训练导入会保留 Backbone 权重，并根据数据集类别配置分类头。查看 Notes 了解分类头初始化情况。选择 torchvision 预训练 ViT 时按其 224×224 输入合同准备模型和数据。

Hugging Face 适配器根据公开仓库 ID 读取配置，并使用 safetensors 权重。导入日志记录下载、构建、权重加载和输入输出检查的进度。

19.3 导入 → 局部替换 → 搜索

以 ResNet 的图像特征 Stage 为例：

1. 导入 ResNet，查看原始模块层级，选择一个具有图像特征图输入输出的 EcosystemBlock。

2. 点击 **Preview searchable replacement**。系统构造原生瓶颈模块并检查输出 shape 与原边界一致。

3. 预览里核对替换位置、通道与下采样；确认后应用。

4. 为暴露的 width 设置如 16、32、64 的离散搜索值，其他节点保持 Fixed。

5. 使用实际延迟、吞吐和显存测量筛选候选；实例化后再训练、评估和导出。

自动替换以图像特征的输入输出 shape 为合同，创建可编辑的瓶颈模块。其他部分保留原始权重，新瓶颈在后续训练中学习任务特征。

19.4 使用硬件测量筛选生态模型

对生态模型和自定义算子，使用 Hardware-aware 搜索记录目标设备的延迟、吞吐和显存，并据此设置候选筛选条件。相关导出保存模型资产、权重哈希、来源与合同，方便后续重建。

20. Code → Visual 与 Architecture Diff

20.1 导入 PyTorch 模型结构

Code → Visual 解析 nn.Module / nn.Sequential 的静态层声明和 forward 数据流，生成可以继续编辑的 Model IR。将模型定义粘贴到导入面板，设置输入类型、shape 和类名后生成预览。

导入器识别层构造、中间变量、模块调用、加法连接和保留 batch 的 Flatten，并把结构信息映射到节点与连线。成熟模型和预训练权重可通过 Model Ecosystem 导入。

20.2 可以直接尝试的模型代码

```
import torch
from torch import nn

class TinyClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(3, 8, 3, padding=1)
        self.relu = nn.ReLU()
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.flat = nn.Flatten()
        self.head = nn.Linear(8, 3)

    def forward(self, x):
        x = self.relu(self.conv(x))
        x = self.pool(x)
        x = self.flat(x)
        return self.head(x)
```

输入 shape 选择 [3,32,32]，不含 batch；输出是每个样本 3 个分类 logits。原始导入示例放在附带 examples 中。操作顺序为：粘贴代码 → 设置 Input type / Shape / Class name → Import / Preview → 检查分析结果 → 应用到画布 → 保存 → 小批次验证。若数据集不是 3 类，应用前修改分类头。

20.3 两个版本怎样比较

在 Architecture Diff 选择左右两个图或历史版本，查看新增/删除/改变的节点、配置和连线。结构 Diff 使用节点、配置和连线记录变化。最好使用同一模型的连续版本，保留节点 ID；若两张完全无关的图节点 ID 不同，看到大量新增/删除是正常的。

候选实例化前保存模板版本。候选应用后进行 Diff，可以检查搜索到底修改了哪个模块，是否误动固定区域。完成 Diff 后，可以继续运行结构检查和小批次验证。

21. Custom Operator SDK：自己的算子进入组件库

21.1 最小可用算子

```
import torch
from torch import nn

class Operator(nn.Module):
    def __init__(self, channels=8):
        super().__init__()
        self.gain = nn.Parameter(torch.ones(1, channels, 1, 1))

    def forward(self, x):
        return x * torch.sigmoid(self.gain)
```

注册参数为 {"channels":8}，Probe shape 为 [8,16,16]，不含 batch。该算子保留 shape；插入位置也必须是 8 通道、16×16 特征图。

21.2 注册与使用步骤

1. 在算子注册面板填写名称、源码、参数 JSON 和 Probe shape。

2. 阅读源码，并明确勾选本地可信代码执行确认。只有你信任的代码才注册。

3. 执行验证。Worker 会构造算子，运行前向、有限数值检查和反向传播。

4. 验证通过后，在组件库查找算子名称并插入合适位置。

5. 在小模型上做 Debug / Profile，再进入搜索或训练。

6. 修改源码或注册参数时创建并验证新版本，不直接篡改资产目录。

注册前审阅算子源码并确认执行。独立 Worker 使用当前用户权限构建模块、运行前向与反向检查，并保存验证合同。

21.3 合同与版本配置

源码上限 50KB；Probe shape 为 1–3 个维度，每维 1–1024，元素总数不超过 1,000,000。返回值必须是单个有限 Tensor，保留 batch 维并支持微分；探测输出最多 4,000,000 个元素，参数最多 50M。

注册版本绑定源码、构造参数和 shape 合同。修改后重新验证并注册新版本，再在模型中选择对应资产。禁用状态用于管理组件的可用性。

22. Experiment Lab：比较、记录与导出

22.1 实验记录包含什么

每个任务保存提交时的模型、数据集 ID、处理流程、设备、种子和配置快照，以及状态、日志、测量与输出文件。修改画布不会把已经提交的任务改成新模型。需要使用新版本时，应再提交任务。

同一项目可有多次训练；比较时核对 Dataset、Split、Seed、Epoch、Batch、AMP、优化器以及设备。结合 Accuracy、资源、训练预算与代理信号并排比较候选。

22.2 导出的四类东西

交付物	用途	能否直接推理
模型结构 / PyTorch 代码 / 配置	重建模型与研究结构	需要相应权重和预处理
训练任务 ZIP	保存训练配置、日志、checkpoint 和可用模型文件	按其中 README、权重和依赖重建
项目 / 模块 / 生态资产引用快照	复用模型来源与模块	仍需检查输入合同和运行依赖
SVG / PNG / PDF / Scene JSON	论文、展示和继续编辑图	保存架构图形与编辑布局

引用生态模型或自定义算子的导出会附带快照与适配代码。下载后检查文件清单，保留 ZIP 原件；完整工作区备份按第 27 章操作。

22.3 Model Optimization 的位置

Optimization Studio 提供幅值剪枝、CPU Linear 动态 INT8 和 ONNX 导出，结合 ONNX Runtime 运行推理检查。选择训练权重和优化设置，生成优化资产，再用固定测试集和目标设备比较精度、体积与速度。

23. Visual Data Pipeline：预处理与增强

23.1 算子参考

操作	参数与默认值	默认生效范围	行为
flip	probability=0.5	train	按概率水平翻转
rotate	quarter_turns=1	train	按 90° 的整数倍旋转
brightness	factor=1.2	all	调整亮度并限制到 0–1
contrast	factor=1.2	all	按图像空间均值调整对比度
noise	std=0.03	train	添加高斯噪声并限制到 0–1
cutout	size=8	train	随机位置置零方块
crop	padding=4	train	边缘填充后随机裁回原尺寸
normalize	mean=0.5、std=0.5	all	按标量均值、标准差归一化
resize	size=32	all	双线性缩放至正方形

Train 表示训练/代理的数据路径可使用该训练转换，验证和测试跳过 train-only 步骤；All 表示两个阶段均使用。不要凭名称猜范围，应检查每个节点的 Scope。尤其 brightness、contrast 的默认值是 all，若只想增强训练集，应显式改为 train。

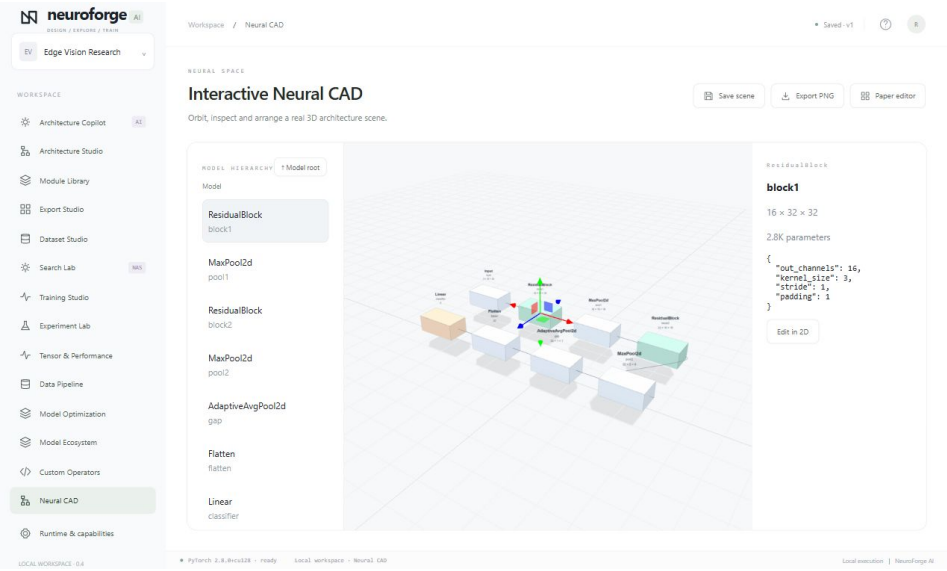
23.2 编辑流程

在 Data Pipeline 添加步骤并拖动调整顺序，编辑参数和 Scope，查看 Before / After 预览。保存并应用后，新任务使用对应的处理快照和随机设置。

图像处理流程支持最多 16 步，输入为 N,C,H,W 浮点 Tensor。normalize 使用标量 mean / std；resize 与模型输入 H/W 保持一致，调整后查看数据预览。

一般把会裁剪到 0–1 的亮度、噪声等操作放在 normalize 前面，否则后续 clamp 可能破坏已归一化数据。每次改变增强后先预览几类样本，再跑短训练；不要只观察一张好看的图片。

24. Export Studio、Figure Studio 与 Neural CAD



交互式 Neural CAD 场景

24.1 一键导出 2D 论文图

1. 保存并检查当前架构，进入 Export Studio / Export 2D。

2. 选择论文风格、方向、尺寸和显示项，如 Labels、Shapes、Params。

3. 生成预览，检查分支、Stage、重复次数和输入输出标注。

4. 论文优先导出 SVG；需要打印或直接提交时可用 PDF，演示和位图排版可用 PNG。

5. 图很长时，先在模型中合理封装模块，或使用竖向布局，不必强行缩成一条极细的横线。

SVG 保留矢量元素，可进一步编辑；PDF 是矢量输出时仍应在最终排版软件中检查字体。不要用网页截图代替原始导出。静态分析不可用的指标不要在论文中改写成真实测量值。

24.2 生成 3D 展示图

选择 3D 模式和 Modern Product / White Minimal / Dark Tech 等风格，设置相机角度、深度、阴影和画面密度。先生成默认构图，再调整模块层次、布局和注释，制作演示、封面与产品展示图。

导出 3D 图有两个入口：在 Figure Studio 编辑投影后的矢量图元，或在 Neural CAD 中旋转和平移 WebGL 场景并导出 3200×2000 PNG。分别保存 Figure 文档和 CAD Scene，即可继续各自的编辑流程。

24.3 Figure Studio 2.0 自由编辑

生成后进入 Figure Studio，选择文字、卡片、连接线或投影面，可移动和修改文字、颜色、尺寸。多选后分组；复制、删除、撤销可用来制作局部放大图和注释。保存 Figure / Scene JSON 后可继续编辑，再导出 SVG / PNG / PDF。

Figure 的文字变化不会改动真实网络的层名、shape 或参数。若后来修改了模型，旧 Figure 仍是旧图形文档；应从新模型重新生成，或自行核对更新。提交论文前逐一确认标题、标签和结构与最终 checkpoint 相符。

24.4 Neural CAD 的场景操作

使用鼠标轨道旋转、平移与缩放；点击模块选中，再拖动平移操纵器安排位置。原生 Module 可深入层级，沿面包屑返回。保存相机和模块位置到项目 Scene；重新打开项目可恢复布局。受保护模型内部模块树以只读结构查看，不会伪装成可任意改写的原生算子。

浏览器禁用 WebGL、显卡驱动异常或远程桌面环境限制时，先使用 2D / Figure Studio 输出。Neural CAD 依赖本地打包的 Three.js，不需要为了显示场景访问外部 CDN。

25. Python SDK、CLI 与 API

25.1 使用包内解释器

以下为 Windows PowerShell 命令。在解压后的 NeuroForge 文件夹打开终端，先让桌面启动器保持服务运行。在同一个 PowerShell 窗口先定义 $nf，后续命令继续使用它：

```
$nf = '.\_internal\python\python.exe'
& $nf -m neuroforge --help
& $nf -m neuroforge system
& $nf -m neuroforge projects
& $nf -m neuroforge datasets
```

如果服务在 8877，--url 放在子命令前：

```
& $nf -m neuroforge --url http://127.0.0.1:8877 system
```

这些命令是运行中服务的客户端，不会自动启动服务。不要把项目名称当成 ID；从 projects 输出复制真正的 id。JSON 配置文件使用 UTF-8。

25.2 提交任务、查询和导出

将下面的 PROJECT_ID 和 JOB_ID 替换为实际 ID：

```
& $nf -m neuroforge proxy --project PROJECT_ID --device auto --config '.\docs\examples\proxy-config.json' --wait
& $nf -m neuroforge train --project PROJECT_ID --device auto --config '.\docs\examples\train-config.json' --wait
& $nf -m neuroforge job JOB_ID
& $nf -m neuroforge control JOB_ID pause
& $nf -m neuroforge control JOB_ID resume
& $nf -m neuroforge export JOB_ID --output '.\my-training.zip'
```

--device 选择实际运行设备；--wait 等待任务进入 completed / failed / cancelled / paused。提交后可通过 job 命令查看状态、日志和输出。

25.3 Python SDK 示例

```
from neuroforge.sdk import NeuroForge

nf = NeuroForge('http://127.0.0.1:8787')
print(nf.system())
projects = nf.projects()
# 按名称选择要运行的项目。
project = next(p for p in projects if p['name'] == 'My first CNN')
job = nf.train(project['id'], epochs=3, batch_size=16, device='auto')
finished = nf.wait(job['id'], timeout=1800)
print(finished['status'], finished.get('error'))
if finished['status'] == 'completed':
    nf.export(job['id'], 'training-result.zip')
```

把代码保存为 .py 文件，再用包内 python.exe 执行。SDK 负责提交项目快照、查询和导出，不在 IDE 进程里直接训练。外部 Python 项目使用 SDK 时可安装源码包或复用 sdk.py 的轻量客户端；计算依赖仍在服务端。

25.4 结构化计划也可以先预览

附带的 profile-plan.json 是一个只请求真实 Profile 的计划示例。先用 preview-plan 审阅，预览不会直接提交计算任务：

```
& $nf -m neuroforge preview-plan '.\docs\examples\profile-plan.json' --project PROJECT_ID
```

SDK 的 apply_plan 需要预览返回的 ID、对应 IR / Search Space 和显式 confirm。若项目变更导致计划过期，应重新预览。自定义算子注册必须传 trusted_code=True / --trust-code；这代表你确认运行其本地 Python 代码。

在服务地址的 /docs 查看交互式 API 文档。服务默认监听 127.0.0.1，供本机工作区与 SDK 使用；API Key 在 Copilot 配置页面录入和管理。

26. 故障排查：从现象找到处理步骤

现象	常见原因	推荐处理
双击没有界面或提示缺少文件	只拷贝 EXE、未完整解压、依赖被删除	重新完整解压，保留 _bootstrap 和 _internal；查看启动器或系统提示
安全软件阻止执行	本地构建未签名或具体文件被判定	不关闭防护、不加白名单；核对来源和系统安全记录，联系维护者提供签名或修复构建
8787 已占用且不是 NeuroForge	其他软件监听该端口	用 NeuroForge.exe --port 8877；不结束不认识的进程
浏览器显示的是旧项目	连接到已经运行的旧服务或选择旧数据目录	看启动器 Data folder、端口、API system 版本；停止自己管理的旧服务后重连
新便携版里看不到旧数据	默认用户数据目录与源码 runtime 不同	按第 4 章选择原 runtime；先备份，不要合并两个 SQLite 文件
正在启动，短时间无网页	首次加载 PyTorch / GPU 较慢	保持启动器，查看数据目录 logs/server.log；启动超时再处理日志中的原因
CUDA unavailable	CPU 环境、驱动或设备不可用	用 auto / cpu 运行；需要 GPU 时检查系统驱动和 /api/system，不要反复重装项目数据
CUDA OOM	激活、batch 或候选模型太大	缩小 batch / 输入 / 模型；开启适用的 AMP 或 Auto batch；保留失败日志，之后重跑
CUDA driver / DLL 错误	驱动与 CUDA runtime 不兼容或包不完整	检查系统驱动与完整包；不要从不明网站下载单个 DLL 放入程序目录
tensor device mismatch	自定义模块创建了固定 CPU Tensor	在算子中使用输入 device 或注册 buffer，重新注册验证新版本
Shape invalid / Linear 不匹配	通道、Flatten、Head 或残差维度错误	回到上一个正确节点，查看实际 shape；Add 两端维度必须兼容
embed_dim must be divisible	hidden dim 与 heads 不整除	调整维度或候选组合；不要把非法组合强行实例化
请求 100 个只得到 27 个	可搜索空间只有 27 个唯一合法组合	查看 Summary；增加变量或值，或接受实际枚举规模
没有候选通过	硬约束过紧、组合非法或测量失败	先用小空间和宽松约束定位原因，再逐步收紧
代理 N/A / failed	结构、激活或数据条件不适用	看单代理失败原因，改用适配代理；不可把 N/A 作为 0 分
MACs / FLOPs 不可用	生态块或自定义块为受保护内部	改用真实延迟 / 显存 / 吞吐约束，参阅 19.4
pause 后没有立即 paused	当前 epoch 还没结束	等待安全点保存；查看 pausing，不要直接杀 Worker
恢复 checkpoint 失败	模型、数据、文件或环境变了	恢复原配置和资产；需要新结构就新建训练任务

现象	常见原因	推荐处理
Key 配置后 HTTP 401 / 403	Key、权限或模型访问错误	检查所选服务的 Key 和模型授权，不把 Key 发到日志反馈
HTTP 404 / 不支持 tools	Base URL / 协议 / 模型不匹配	确认 Base URL 只到 /v1 等服务根；选匹配协议和支持结构化工具的模型
Copilot timeout	网络、服务响应或预算超时	查看可读错误，简化请求后重试；已有模型和手动功能仍可使用
计划提示 stale / expired	预览后项目、数据或搜索空间已变	以最新上下文重新生成并预览，不复用旧批准
模型下载失败	远端不可达、资源缺失或没有下载确认	先用不下载的原生模板；核对公开仓库和允许下载设置
Figure 与当前模型不一致	图形文档是旧架构的独立快照	从当前模型重新生成，再补充注释
没有完整 Tensor 文件	超出保存预算或任务未完成	查看 Debug 的数组记录与预算说明；减小 batch / 模型后重新采集
停止后台被拒绝	有 queued / running 等活动任务	在对应页面暂停或取消任务，等待稳定状态后再 Stop backend

26.1 发给维护者的最小诊断信息

记录程序版本、操作步骤、任务 ID、错误文字、Device、数据 shape、选择的模型、相关日志时间段。可以附不含隐私数据的模型 IR 和配置。不要发送 API Key、Cookie、完整私有数据集或未经确认的敏感权重。

启动器日志位于当前数据目录的 logs/server.log，任务日志在各自任务记录中。接口返回错误应保留原文；先定位第一个真实异常，再处理连带失败。不要为“清理错误”删除整个数据目录。

27. 备份、迁移、升级与卸载

27.1 完整备份步骤

1. 在界面确认没有 queued / running / pausing / cancelling 任务；正在训练时先暂停并等待 checkpoint 保存。

2. 用启动器 Stop backend 停止自己管理的服务。

3. 完整复制当前 Data folder 到带日期的备份位置，包括 SQLite、可能存在的 WAL / SHM、数据集文件、任务目录、模型和自定义算子资产。

4. 另外保存重要训练 ZIP、原始数据和源代码。确认备份空间足够，不要只复制数据库文件。

5. 恢复时选择一份完整备份副本作为 Data folder，启动后先看项目、任务、数据集是否齐全。

SQLite 使用的 WAL / SHM 等文件与数据库一起保存在数据目录。停止服务后复制整个目录，可保留数据、权重、资产和实验记录。

27.2 升级的做法

把新程序解压到一个新文件夹，先阅读版本说明、备份用户数据，再用新启动器选择原数据目录。不要在正在运行的 _internal 里覆盖 DLL。不要让两个后端同时读写同一个数据目录。跨版本升级前保留完整备份，方便选择对应版本继续实验。

启动器的“关闭但保留后台”会让服务继续运行；此时程序占用可能仍存在。先真正停止服务再移动程序文件夹。API Key 只在后台 RAM 中，关闭浏览器不一定清掉 Key；Disconnect 或停止后台会清掉，最长会话约八小时。

27.3 卸载

停止后台后可以删除程序目录，用户数据与导出文件可分别保留。默认桌面数据位于 %LOCALAPPDATA%\NeuroForgeAI；迁移或清理前可先制作完整备份。

28. 三条完整实战路线

28.1 自己的图像分类数据 → NAS → 训练

准备每类若干图片并按 7.1 打成 ZIP；导入、查看质量提示并保存划分；应用到项目。选择小型 CNN，调整 Head 为实际类别数。只搜索最后一个 Block 的宽度和重复次数，保留固定 Stem。先检查合法组合，再启动 8–16 个候选的小预算搜索。查看代理和硬件结果，挑选代表性候选实例化，先训练 3 个 epoch 检查流程，再按实验需求增加预算。

完成后保存五类结果：数据版本和划分、Search Space 快照、最终固定模型、训练记录与 checkpoint、导出 ZIP。最后从选定架构生成论文图和展示图。

28.2 导入 ResNet → 局部替换 → 比较

从 Model Ecosystem 导入受支持 ResNet，先保存原始版本并在目标设备记录基线。选择一个支持合同的 Stage，预览可搜索替换并应用。为 width 定义候选，进行硬件感知搜索。保存新候选后以匹配协议训练；在 Diff 看结构变化，在 Experiment Lab 看训练与延迟，最后比较实际准确率和成本。保存评测与延迟结果，形成完整的优化记录。

28.3 ViT → Copilot → 人工批准

从原生 ViT 模板开始，先检查 Patch 数量、Embedding 维度和 Head。让 Copilot 把 Encoder depth 设为 4–12 的离散范围，heads 设为 4、8、16，参数预算 50M；说明想用 Bayesian GP-UCB 和 CUDA 测量。审阅计划中真实节点路径、变量、约束和任务数量。若只有 27 个唯一组合，系统最多探索合法空间，不会凭空生成 100 个不同候选。确认后运行，选择候选固定并真实训练。

任一路线出错时都可以回到手动编辑：AI 负责提出结构化计划，确定性的检查器与 Worker 决定配置能否执行。

29. 参数配置与术语速查

29.1 本地运行预算

对象	配置参考
顶层 Model IR	1–256 个节点；嵌套展开另有预算
编译分析预算	展开节点最多 2048；参数 250M；激活元素 32M；MACs 10G（适用的原生静态分析）
搜索任务	候选预算最多 256，实际唯一合法候选可能更少
Pipeline	最多 16 步；图像 Tensor 路径
自定义算子	50KB 源码、50M 参数与受限探测 shape，见第 21 章
Figure 文档	最多 12,000 个图元；用于控制本地图形编辑规模
模型生态	按适配器、输入合同和 250M 资产预算配置模型
CUDA 设备	按任务选择可见的 cuda:N
停止与恢复	在 epoch 安全点保存并恢复训练状态

结合输入、batch、激活、优化器状态和目标设备设置运行预算。需要扩大模型时，先逐步调整 batch 和结构，再用 Profiler 查看资源变化。

29.2 常用术语

术语	在本程序中的含义
IR	独立于画布坐标的模型结构表示
Model Graph	记录算子与 Tensor 的依赖关系
Search Space	变量、条件和约束；实例化后才是固定模型
Proxy	用于架构排序的低成本代理信号
Candidate	搜索生成的一份具体结构与对应测量
Checkpoint	训练恢复所需状态；与只用于推理的权重有区别
AMP	在适用 GPU 路径使用自动混合精度
Micro-batch	单次前向/反向实际处理的样本数
Accumulation	多个微批次累计梯度后再进行优化器更新
MACs / FLOPs	计算规模指标，需要说明统计口径

术语	在本程序中的含义
Allocated / Peak	PyTorch 分配显存及任务峰值口径，不等同整个 GPU 的全部占用
Contract	模块允许的输入输出类型和 shape
Asset	带元数据、版本或校验的权重、模块或算子资源
Scene	可编辑图形 / 3D 布局文档，与真实模型结构分开保存
BYOK	使用你自己的模型服务 API Key
Pareto	多指标下没有被另一候选同时全面胜过的候选集合

30. 验收清单、文件索引与参考

30.1 第一次使用后的操作清单

• 双击 EXE 后能打开启动器与本地界面，无需安装 Python。

• System 中看见真实设备；选择 CPU 或 CUDA 后，任务记录显示实际使用设备。

• 图结构检查通过，Proxy 与短训练均完成，有日志、指标和 checkpoint。

• Search Space 只修改选定区域；实例化后可作为固定模型继续训练。

• 停止后台前活动任务受到保护；重新打开程序后，旧数据仍可见。

• 导出模型、实验 ZIP 和图形文件，在程序目录外打开并检查内容。

30.2 随包文件索引

NeuroForge-User-Manual-zh-CN.html 为可搜索网页版说明书，左侧目录可跳转章节；.pdf 为适合离线阅读和打印的固定排版版；.md 是可自行修订的文本源。三版内容来自同一份正文。

examples 目录提供 Token JSON、训练/代理配置、TinyClassifier 导入代码、可注册 Operator 和 Profile 计划。带 PROJECT_ID / JOB_ID 的命令需要替换真实 ID；样例只描述最小流程，可以作为你自己的任务配置起点。

源码目录包含应用、桌面构建脚本、测试和示例。便携构建的 BUILD-MANIFEST.json 记录版本与入口哈希，PORTABLE-VERIFICATION.json 记录构建检查结果。

30.3 参考资料与阅读顺序

进一步理解底层工具可阅读官方文档：

• PyTorch：https://pytorch.org/docs/stable/index.html

• torchvision 模型：https://docs.pytorch.org/vision/stable/models.html

• timm：https://huggingface.co/docs/timm/index

• Transformers：https://huggingface.co/docs/transformers/index

• PyInstaller 打包模式：https://pyinstaller.org/en/stable/operating-mode.html

• CPython Windows 运行：https://docs.python.org/3.10/using/windows.html

新手先完成第 2–6 章，再按任务阅读第 7–13 章；现代架构与模型导入见第 15–20 章，论文图与 3D 场景见第 24 章，日常维护和故障排查见第 26–27 章。